ЛР 2. Твой код простыми словами

Светофор с командами через UART: какой файл открыть, что делает код и как команда с компьютера меняет работу программы.

Все пути - внутри `lab2`. Файлы `.c` из таблицы лежат в `Core/Src`, файлы `.h` - в `Core/Inc`. Номера в блоках - строки исходников на 03.11.2026; длинные строки перенесены.

Карта основных файлов

Файл	За что отвечает
`main.c`	Запускает всё и по очереди обслуживает связь, команды, кнопку и светофор.
`console.c`, `console.h`	Собирают символы в строку, возвращают эхо и запускают готовую команду.
`command.c`, `command.h`	Распознают текст: что именно попросил пользователь и верны ли аргументы.
`serial.c`, `serial.h`	Хранят входящие и исходящие байты в очередях; переключают опрос и прерывания.
`uart_backend.h`	Перечисляет общие функции, через которые `serial.c` обращается к UART.
`uart_hal.c` `uart_register.c`	Два варианта UART: первый вызывает HAL, второй сам работает с регистрами контроллера.
`uart_pins.c`	Настраивает выводы PA9 и PA10 для USART1.
`traffic.c`, `traffic.h`	Правила светофора, длительность красного и режим обработки кнопки.
`board.h`	Выводы платы, время проверки кнопки и скорость UART.
`button.c`, `led.c`, `gpio_driver.c` и их `.h`	Обработка кнопки, управление лампами и настройка выводов, как в ЛР 1.

Путь одной команды

UART принимает байты → `serial.c` складывает их в очередь → `console.c` собирает строку до Enter → `command.c` определяет команду → `console.c` вызывает нужную функцию и ставит ответ в очередь отправки.

В `.h` описано, какие функции и данные доступны. В `.c` написаны действия этих функций.

1. main.c: что запускается и повторяется

Что добавилось при запуске

Core/Src/main.c, строки 58-62

```
58      traffic_init(&traffic, now);
59      if (!serial_init(true)) fatal(7);
60      console_init(&console);
61      static const char greeting[] = "\r\nLab2 traffic: 57600 8N1, mode 1, timeout 12,
    I\r\n";
62      (void)serial_write(greeting, sizeof(greeting) - 1U);
```

Сначала запускается светофор. `serial_init(true)` включает UART в режиме прерываний. `console_init` очищает состояние консоли. Затем приветствие ставится в очередь отправки через `serial_write`.

Начало главного цикла

Core/Src/main.c, строки 63-68

```
63      for (;;) {
64          serial_service();
65          console_update(&console, &traffic);
66          now = HAL_GetTick();
67          bool press = button_update(&button, !io_read_pin(BOARD_BUTTON_PORT,
    BOARD_BUTTON_PIN), now);
68          traffic_update(&traffic, press, now);
```

64: обслужить UART и отправку. **65:** разобрать принятые символы и команды. **66-68:** взять время, проверить кнопку и обновить светофор. Поэтому изменение настроек командой может повлиять на светофор в этом же проходе цикла.

Ниже, в строках **69-76**, `traffic_light` выбирает цвет, `switch` переводит его в `BOARD_RED`, `BOARD_GREEN` или `BOARD_YELLOW`, а `led_write` управляет лампами.

Где искать основные настройки

Настройка	Файл и строки
Выводы ламп PD13/PD14/PD15, кнопка PC15 и проверка нажатия 30 мс	`Core/Inc/board.h`, 5-12
Скорость UART: 57600	`Core/Inc/board.h`, 13
Начальные времена: красный 12 с, зелёный 3 с, мигание 3 с, жёлтый 1 с	`Core/Inc/traffic.h`, 5-6
Размер очередей: приём 128 байт, отправка 512 байт	`Core/Inc/serial.h`, 6-11
Строка 64 байта, из них до 63 символов команды; запас для ответа 96 байт	`Core/Inc/console.h`, 5

`SystemClock_Config` (22-43) задаёт частоту 16 МГц. `SysTick_Handler` (80) обновляет счётчик времени. `fatal` (16-20) сохраняет ошибку в `lab2_fault` и останавливает работу.

2. console.c: из символов получается строка

Здесь хранится то, что пользователь набирает. В `Console` есть `line` - текст, `length` - его длина, `discard` - признак испорченной строки, `after_cr` - обработка CRLF и `switching` - ожидание смены режима UART (`console.h`, 6-11).

Консоль берёт символы небольшими порциями

Core/Src/console.c, строки 44-46

```
44      for (unsigned i = 0; i < 16 && serial_tx_free() >= CONSOLE_REPLY_RESERVE; ++i) {
45          uint8_t byte;
46          if (!serial_read(&byte)) return;
```

За один вызов разбирается до 16 байт. Если входная очередь пуста, функция сразу возвращается. Перед чтением также проверяется место для ответа: это помогает не потерять эхо и ответ, когда отправка занята.

Эхо, Enter и удаление последнего символа

Core/Src/console.c, строки 54-64

```
54          (void)serial_write(&byte, 1);
55          if (byte == '\n' && c->after_cr) { c->after_cr = false; continue; }
56          c->after_cr = byte == '\r';
57          if (byte == '\r' || byte == '\n') {
58              if (c->discard) reply("\r\nERROR: input lost or line too long; repeat
    command\r\n");
59              else if (c->length) execute(c, t);
60              c->discard = false;
61              c->length = 0;
62              if (c->switching) return;
63          } else if (byte == 8 || byte == 127) {
64              if (!c->discard && c->length) --c->length;
```

54: отправить полученный символ обратно в терминал - это эхо. **55-56:** считать пару CRLF одним завершением строки. **57-62:** по Enter выполнить непустую команду либо сообщить об испорченном вводе, затем очистить строку.

63-64: Backspace или Delete уменьшают `length` на один. Последний набранный символ больше не входит в команду.

Где обычный символ добавляется в текст

Core/Src/console.c, строки 65-69

Печатные символы добавляются в `line`, пока есть место. Слишком длинная строка или неподдерживаемый управляющий символ включают `discard`. Такая строка не выполняется; после Enter пользователь получает сообщение и может повторить ввод.

Как консоль узнаёт о потерянных данных

В строках **33-38** и **48-53** проверяется `rx_epoch` - номер случая потери входных данных. Если он изменился, накопленная команда сбрасывается и отвергается до Enter. Так случайно обрезанная команда не запускается.

3. command.c: какую команду набрали

`command_parse` возвращает `Command`: `type` - что сделать, `value` - число. Описание этих данных - в `Core/Inc/command.h`, строки 4-5.

Простые команды сравниваются с готовым текстом

Core/Src/command.c, строки 7-11

```
 7      if (!strcmp(line, "?")) return (Command){CMD_STATUS, 0};
 8      if (!strcmp(line, "set mode 1")) return (Command){CMD_MODE, 1};
 9      if (!strcmp(line, "set mode 2")) return (Command){CMD_MODE, 2};
10      if (!strcmp(line, "set interrupts on")) return (Command){CMD_INTERRUPTS, 1};
11      if (!strcmp(line, "set interrupts off")) return (Command){CMD_INTERRUPTS, 0};
```

`!strcmp(...)` означает «строки совпали»: при совпадении `strcmp` возвращает 0. Регистр и пробелы учитываются; `set interrupts ON` даст ошибку.

Как из set timeout 8 получается число 8

В строках **12-15** код узнаёт начало `set timeout` и переходит к цифрам после него. Пустое число отвергается.

Core/Src/command.c, строки 17-20

```
17              if (*p < '0' || *p > '9') return (Command){CMD_ERROR, 0};
18              unsigned digit = (unsigned)(*p - '0');
19              if (value > (MAX_TIMEOUT_SECONDS - digit) / 10U) return
    (Command){CMD_ERROR, 0};
20              value = value * 10U + digit;
```

Разрешены только цифры. Строка 20 собирает число по одной цифре: старое значение умножается на 10 и к нему добавляется новая цифра. Проверка перед этим не даёт выйти за максимум **86400 секунд**. Ноль тоже отвергается в строке 22.

Где распознанная команда выполняется

Core/Src/console.c, строки 22-23

```
22      case CMD_MODE: (void)traffic_set_mode(t, (uint8_t)command.value);
    reply("\r\nOK\r\n"); break;
23      case CMD_TIMEOUT: (void)traffic_set_timeout(t, command.value);
    reply("\r\nOK\r\n"); break;
```

`execute` в `console.c` вызывает нужную функцию и отвечает `OK`. Ниже, в строках 24-27, команда смены UART только запрашивает переключение. Ответ на неё отправляется после перехода в новый режим.

Где формируется ответ на вопросительный знак

В `console.c`, строки **14-20**, `snprintf` собирает ответ вида `red, mode 1, timeout 12, I`. `traffic_name` даёт название фазы; тайм-аут выводится в секундах. `I` - прерывания, `P` - опрос. Для неверной команды строка 28 отправляет `ERROR`.

4. traffic.c: настройки светофора

Что хранит объект Traffic

Core/Inc/traffic.h, строки 9-14

`state` - текущая фаза: красный, зелёный, мигание или жёлтый.
`since` - время начала текущей фазы.
`red_ms` - выбранная длительность красного в миллисекундах.
`request` - просьба сократить красный.
`mode` - 1: учитывать кнопку; 2: игнорировать кнопку.

Где применяется set timeout

Core/Src/traffic.c, строки 6-10

```
 6  bool traffic_set_timeout(Traffic *t, uint32_t seconds)
 7  {
 8      if (!seconds || seconds > MAX_TIMEOUT_SECONDS) return false;
 9      t->red_ms = seconds * 1000U;
10      return true;
```

Команда передаёт секунды; умножение на 1000 переводит их в миллисекунды. `since` здесь не меняется: отсчёт текущего красного продолжается от его прежнего начала.

Где применяется set mode

Core/Src/traffic.c, строки 12-17

```
12  bool traffic_set_mode(Traffic *t, uint8_t mode)
13  {
14      if (mode != 1 && mode != 2) return false;
15      t->mode = mode;
16      if (mode == 2) t->request = false;
17      return true;
```

Допускаются только режимы 1 и 2. При переходе в режим 2 также удаляется сохранённая просьба от кнопки.

Где кнопка влияет на красный

Core/Src/traffic.c, строки 22-25

```
22      if (press && t->mode == 1 && t->state != TRAFFIC_GREEN) t->request = true;
23      uint32_t duration;
24      switch (t->state) {
25      case TRAFFIC_RED: duration = t->request ? t->red_ms / 4U : t->red_ms; break;
```

Нажатие принимается в режиме 1, кроме обычного зелёного. Если просьба есть, красный длится `red_ms / 4`, иначе - всё `red_ms`. При тайм-ауте 8 с сокращённый красный равен 2 с от начала фазы.

Переходы по времени - строки 30-33; выбор цвета и мигание - 35-45; текстовое название фазы для статуса - 46-55. Исходный красный и режим 1 задаёт `traffic_init` в строке 4.

5. serial.c: очереди входа и выхода

Этот модуль работает с байтами. Один байт здесь обычно соответствует одному символу терминала. RX означает приём, TX - отправку.

Какие данные лежат в serial.c

Core/Src/serial.c, строки 5-9

Переменная	Простой смысл
`rx[]` / `tx[]`	Массивы для принятых байтов и байтов, которые ещё нужно отправить.
`rx_head`, `rx_tail` `tx_head`, `tx_tail`	Места добавления и извлечения. Дойдя до конца массива, индекс возвращается в начало.
`rx_count` / `tx_count`	Сколько байтов сейчас в каждой очереди.
`tx_byte`, `busy`	Текущий передаваемый байт и признак, что его передача ещё занята.
`irq_mode`, `target_mode`	Режим, который уже работает, и режим, на который попросили перейти.

serial_write складывает весь ответ в очередь

Core/Src/serial.c, строки 26-33

```
26      if (size > SERIAL_TX_CAPACITY - tx_count) return false;
27      const uint8_t *bytes = data;
28      for (size_t i = 0; i < size; ++i) {
29          tx[tx_head] = bytes[i];
30          tx_head = (uint16_t)((tx_head + 1U) % SERIAL_TX_CAPACITY);
31      }
32      tx_count += (uint16_t)size;
33      return true;
```

Если места мало, функция возвращает `false` и ничего не добавляет. Если места хватает, копирует весь блок и возвращает `true`. Это постановка в очередь; реальная отправка выполняется отдельно.

Куда попадает принятый байт

`serial_received` (62-64) кладёт байт в RX. `serial_read` (35-45) забирает самый ранний байт и отдаёт его консоли. Короткие `lock` / `unlock` (10-11) защищают общие данные при работе основного кода и прерываний.

Что происходит при переполнении или ошибке

В строках **55-60** при полном RX увеличивается счётчик, меняется `rx_epoch` и входная очередь очищается. `serial_receive_error` (66-72) делает похожий сброс при ошибке приёма. Консоль видит новый `rx_epoch` и отвергает повреждённую строку.

6. serial_service: отправка и смена режима

Функция вызывается каждый проход `main`. Она постепенно двигает обмен вперёд, а управление снова возвращается к командам, кнопке и светофору.

Где применяется запрос смены I/P

Core/Src/serial.c, строки 89-93

```
89      if (serial_switch_pending() && serial_idle()) {
90          uint32_t p = lock();
91          if (uart_backend_mode(target_mode)) irq_mode = target_mode;
92          unlock(p);
93      }
```

Если режим попросили поменять и передача закончилась, вызывается `uart_backend_mode`. `serial_idle` (75) проверяет пустую TX, отсутствие занятого байта и аппаратное окончание передачи. `serial_request_mode` (76) только запоминает пожелание.

Ниже, в **94-98**, режим опроса сам проверяет приём байта. В режиме прерываний байты поступают через обработчик UART.

Как уходит следующий байт в режиме прерываний

Core/Src/serial.c, строки 100-105

```
100     if (!tx_count || busy) return;
101     if (irq_mode) {
102         uint32_t p = lock();
103         tx_byte = tx[tx_tail];
104         busy = true;
105         if (uart_backend_transmit(&tx_byte, true)) {
```

Если TX пуста или предыдущий байт занят, функция возвращается. Иначе запускает передачу; при успехе строки 106-107 убирают байт из TX. По завершении `serial_transmitted` (74) сбрасывает `busy`. Следующий байт запустит новый вызов `serial_service`.

В режиме опроса работает другая ветка, **110-116**: попытаться отправить один байт и удалить его из очереди при успехе.

Почему OK приходит уже после переключения

Core/Src/console.c, строки 39-43

```
39      if (c->switching) {
40          if (serial_switch_pending() || serial_tx_free() < CONSOLE_REPLY_RESERVE)
    return;
41          reply("\r\nOK\r\n");
42          c->switching = false;
43      }
```

После команды смены I/P консоль проверяет, закончилось ли переключение и есть ли место для ответа. Когда всё готово, отправляет OK и продолжает разбор команд. Основной цикл всё это время продолжает работу.

7. uart_hal.c: UART через готовую библиотеку

Эта реализация выбирается в обычной HAL-сборке. Через функции из `uart_backend.h` модуль `serial.c` вызывает её настройку, приём, передачу и проверку завершения.

Где указаны параметры UART

Core/Src/uart_hal.c, строки 14-21

```
14      uart.Instance = USART1;
15      uart.Init.BaudRate = BOARD_UART_BAUD;
16      uart.Init.WordLength = UART_WORDLENGTH_8B;
17      uart.Init.StopBits = UART_STOPBITS_1;
18      uart.Init.Parity = UART_PARITY_NONE;
19      uart.Init.Mode = UART_MODE_TX_RX;
20      uart.Init.HwFlowCtl = UART_HWCONTROL_NONE;
21      uart.Init.OverSampling = UART_OVERSAMPLING_16;
```

Код выбирает USART1, скорость из `BOARD_UART_BAUD`, 8 бит данных, 1 стоп-бит и отсутствие чётности. Включены приём и отправка. `HAL_UART_Init` в строке 24 применяет настройки.

Как один и тот же файл отправляет в I и P

Core/Src/uart_hal.c, строки 51-55

```
51  bool uart_backend_transmit(uint8_t *byte, bool interrupts)
52  {
53      if (interrupts) return HAL_UART_Transmit_IT(&uart, byte, 1) == HAL_OK;
54      if (!__HAL_UART_GET_FLAG(&uart, UART_FLAG_TXE)) return false;
55      HAL_StatusTypeDef result = HAL_UART_Transmit(&uart, byte, 1, 2);
```

При `interrupts = true` вызывается `HAL_UART_Transmit_IT`: запуск передачи с прерываниями. В опросе сначала проверяется готовность, затем `HAL_UART_Transmit` отправляет один байт с тайм-аутом 2 мс. `uart_backend_receive` (26-33) проверяет приём без ожидания нового байта.

Где принятый байт передаётся в serial.c

Core/Src/uart_hal.c, строки 64-71

```
64  void USART1_IRQHandler(void) { HAL_UART_IRQHandler(&uart); }
65  void HAL_UART_RxCpltCallback(UART_HandleTypeDef *handle)
66  {
67      if (handle != &uart) return;
68      if (handle->ErrorCode == HAL_UART_ERROR_NONE) {
69          serial_received(rx_byte);
70          if (irq_enabled) (void)HAL_UART_Receive_IT(handle, &rx_byte, 1);
71      }
```

Прерывание вызывает `HAL_UART_IRQHandler`. После приёма HAL вызывает `HAL_UART_RxCpltCallback`: функция кладёт байт в RX через `serial_received` и сразу заказывает приём следующего байта.

Завершение отправки обрабатывает `HAL_UART_TxCpltCallback` (73-76): вызывает `serial_transmitted`. Ошибки приёма обрабатываются ниже, в `HAL_UART_ErrorCallback` (77-84). Режим I/P настраивает `uart_backend_mode` (34-50).

8. Свой UART: uart_register.c и uart_pins.c

uart_pins.c: выводы для связи

Core/Src/uart_pins.c, строки 5-8

```
5      const IoConfig pins = {IO_ALTERNATE, IO_PULL_UP, 2, false, 7};
6      if (!io_clock_enable(GPIOA) ||
7          io_configure(GPIOA, (uint16_t)((1U << 9) | (1U << 10)), &pins, 0) != IO_OK)
8          return false;
```

Выбраны PA9 и PA10, а число 7 задаёт AF7 - подключение этих выводов к USART1. В строках 10-12 PC6 и PC7 оставлены входами: на этой плате они соединены с теми же линиями.

uart_register.c: настройки без HAL UART

Core/Src/uart_register.c, строки 18-19

```
18     USART1->BRR = (16000000U + BOARD_UART_BAUD / 2U) / BOARD_UART_BAUD;
19     USART1->CR1 = USART_CR1_UE | USART_CR1_RE | USART_CR1_TE;
```

В BRR записывается настройка скорости. В CR1 включаются USART, приём и отправка. Подготовка устройства находится в строках 7-24; выбор I/P - в 25-37.

Где читается входной байт

Core/Src/uart_register.c, строки 40-45

```
40     uint32_t status = USART1->SR;
41     if (!(status & (USART_SR_RXNE | errors))) return 0;
42     uint8_t data = (uint8_t)USART1->DR;
43     if (status & errors) return -1;
44     *byte = data;
45     return 1;
```

SR содержит признаки состояния, DR - данные. Нет байта и ошибок - вернуть 0. Есть ошибка - вернуть -1. Исправный байт - записать его в *byte и вернуть 1. В опросе эти результаты использует serial_service.

Что делает обработчик прерывания при отправке

Core/Src/uart_register.c, строки 67-73

```
67     if ((sr & USART_SR_TXE) && (cr1 & USART_CR1_TXEIE) && pending_tx) {
68         USART1->DR = *pending_tx;
69         pending_tx = 0;
70         USART1->CR1 = (cr1 & ~USART_CR1_TXEIE) | USART_CR1_TCIE;
71     } else if ((sr & USART_SR_TC) && (cr1 & USART_CR1_TCIE)) {
72         USART1->CR1 &= ~USART_CR1_TCIE;
73         serial_transmitted();
```

Если можно записать байт (TXE) и есть pending_tx, код переносит его в DR. Затем ждёт завершения передачи (TC) и вызывает serial_transmitted. Приём в этом же обработчике находится в строках 62-65.

Запуск отправки - строки 47-58: в I запоминается адрес байта и включается прерывание TXE; в P проверяется готовность и байт сразу пишется в DR.

9. Что осталось от ЛР 1 и что помогает запуску

Файл или группа	Где и что делает
`Core/Src/button.c` `Core/Inc/button.h`	Строки 7-18: подтверждает устойчивое положение кнопки и даёт одно событие нажатия. Вызывается даже в режиме 2; режим учитывает уже светофор.
`Core/Src/led.c` `Core/Inc/led.h`	Настройка ламп - 2-9; передача значений в GPIO - 10-13. Цвета выбираются в `main.c`.
`Core/Src/gpio_driver.c` `Core/Inc/gpio_driver.h`	Свой драйвер выводов. Запись - 21-24, чтение одного вывода - 27-30, настройка - 45-96. Его использует и `uart_pins.c`.

`Core/Startup/`
`Core/Src/system_stm32f4xx.c`
`Drivers/`
	Стартовый и системный код, CMSIS и HAL производителя. Подготавливают запуск; вызов `main` - строка 100 стартового файла.

`Core/Inc/`
`stm32f4xx_hal_conf.h`
`STM32F427VGTX_FLASH.ld`
	Настройки библиотеки HAL и правила размещения программы в памяти.

`tools/build.py`	Выбирает HAL или свой UART и собирает обычную либо тестовую прошивку.
`tools/board.py` `SDK11M.cfg`	Проверка подключения, прошивка и отладка через FT2232/JTAG. По умолчанию скрипт берёт `Debug/lab2.elf`.
`tests/test_lab2.c` `tests/run_tests.py`, `test.ld`	Проверки светофора, команд и UART. Python запускает ARM-код в эмуляторе; LD задаёт размещение тестов.

`.project`, `.cproject`
`README.md`, `docs/`
`tools/render_schematic.py`
	Настройки IDE, описание проекта, отчёт и схема. Скрипт делает картинки страниц схемы и не участвует в работе светофора.

Как выбираются две реализации UART

Из корня репозитория:

```
python3 lab2/tools/build.py
python3 lab2/tools/build.py --backend register
```

Первая команда создаёт `build/lab2_hal.elf`, вторая - `build/lab2_register.elf` внутри `lab2`. Вторая задаёт `UART_REGISTER_BACKEND`: поэтому включается код `uart_register.c`, а в обычной сборке - `uart_hal.c`. Выбор указан в `tools/build.py`, строки 27 и 40-41.

HAL / register выбираются при сборке. I / P переключаются командой во время работы в обеих сборках.

10. Что показать на защите

Пример: команда set timeout 8

1. UART принимает символы; `serial_received` кладёт их в RX.
2. `console_update` забирает символы и собирает `set timeout 8`.
3. По Enter `command_parse` возвращает `CMD_TIMEOUT` и число 8.
4. `execute` вызывает `traffic_set_timeout`: `red_ms` становится 8000.
5. OK попадает в TX, а `serial_service` постепенно отправляет его. Если текущий красный уже идёт 8 секунд или больше, следующий вызов `traffic_update` завершит его; отсчёт не начинается заново.

Быстрая подсказка: где что найти

Нужно показать или изменить	Открой
Основной цикл	`Core/Src/main.c`, 63-76
Скорость UART / размеры очередей	`Core/Inc/board.h`, 13 `Core/Inc/serial.h`, 6-11
Enter, эхо, Backspace и длину строки	`Core/Src/console.c`, 44-69
Текст допустимых команд и проверку числа	`Core/Src/command.c`, 4-25
Выполнение команды / ответ со статусом	`Core/Src/console.c`, 9-30
Настройку красного и режима кнопки	`Core/Src/traffic.c`, 6-25
Очередь отправки / извлечение входа	`Core/Src/serial.c`, 21-45
Ожидание и смену I/P	`Core/Src/serial.c`, 75-78, 89-93 `Core/Src/console.c`, 39-43
Выводы UART / обработчик прерывания	`Core/Src/uart_pins.c`, 3-13 `uart_hal.c`, 64-84 `uart_register.c`, 60-75

Как коротко объяснить своё решение

«Я добавил к светофору управление через терминал. UART принимает байты, общий модуль связи хранит их в очереди, консоль собирает строку, а разборщик определяет команду. После этого меняются настройки светофора или связи и отправляется ответ. Один и тот же код команд работает с HAL и с моим драйвером UART».

Памятка составлена по текущим исходникам lab2. Фрагменты взяты из файлов автоматически; исходники при подготовке документа не менялись.